```
        Behaviors.monitor(probe.ref, CounterTimer())            ❶
      val counter = spawn(counterMonitored)                     ❷

      counter ! CounterTimer.Pause(1)                           ❸
      probe.fishForMessage(3.seconds) {                         ❹
        case CounterTimer.Increase =>
          FishingOutcomes.continueAndIgnore
        case CounterTimer.Pause(_) =>
          FishingOutcomes.continueAndIgnore
        case CounterTimer.Resume => FishingOutcomes.complete    ❺
      }
    }
  }
}
```

❶ Uses a probe to monitor messages sent to the counter actor

❷ Spawns the actor under test

❸ Asks to pause for 1 second

❹ Fishes, with a patience of 3 seconds

❺ Completes with success

To skip some work, you can extend from `ScalaTestWithActorTestKit`—as the previous listing does—to avoid creating the `ActorTestKit` and terminating the `ActorSystem` by yourself. Without the reference to the `ActorTestKit`, you can use `spawn` and `createTestProbe` directly instead of `testKit.spawn` and `testKit.createTestProbe` as before. This is possible because the `testKit` is implicitly available within the `ScalaTestWithActorTestKit`.

4.3.3 Logging

Log testing is similar in an asynchronous environment, although there are a couple of differences. Instead of looking at a queue of logs, you